Assignment 1 — Solving the 8-Puzzle with Uninformed and Informed Search

Contents

1 Problem and approach **2**
- 1.1 Assumptions and details . . . 2
- 1.2 Code structure . . . 2

2 Algorithms **2**
- 2.1 Breadth-first search . . . 2
- 2.2 Depth-first search . . . 2
- 2.3 Iterative-deepening DFS (IDS) . . . 2
- 2.4 A* search . . . 3

3 Heuristics: Manhattan vs. Euclidean **4**

4 Results for all algorithms **4**
- 4.1 Observations . . . 5
- 4.2 Paths to the goal (blank moves: U/D/L/R) . . . 5

5 Sample run (step by step) **6**

6 Extra work **7**

7 How to run **7**

1 Problem and approach

The 8-puzzle is a 3×3 board with tiles $1 \ldots 8$ and a blank (written 0). A *state* is a tuple of nine integers read row by row, e.g. `(1,2,5,3,4,0,6,7,8)`. The *goal* is `(0,1,2,3,4,5,6,7,8)`. The four actions `Up, Down, Left, Right` move the **blank** one cell; each action costs 1, so the path cost equals the number of moves.

I implemented the four required algorithms — BFS, DFS, Iterative-Deepening DFS (IDS) and A* — and A* with two heuristics (Manhattan and Euclidean). Everything is written in Python 3 with the standard library only.

1.1 Assumptions and details

- Only half of the 9! boards are solvable. A board is solvable iff the number of inversions (ignoring the blank) is even; the program checks this first and refuses unsolvable input (`is_solvable`).
- Children are generated in the fixed order Up, Down, Left, Right, so every run is deterministic.
- **Counting rule (identical for all algorithms).** *Nodes expanded* increases by one each time a node is taken from the frontier, is not the goal, and its children are generated. The goal test happens when a node is *removed* from the frontier. *Search depth* is the depth of the goal node; *cost* is the length of the returned path; *running time* is wall-clock time measured with `time.perf_counter()` around the search only.
- DFS and BFS use a visited set (graph search) so they terminate. IDS cannot use a global visited set (see §2.3).

1.2 Code structure

`puzzle/state.py`	state representation, neighbours, solvability, random states
`puzzle/result.py`	`SearchResult` record and path reconstruction
`puzzle/bfs.py, dfs.py`	breadth-first and depth-first search
`puzzle/ids.py`	**iterative deepening DFS (new)**
`puzzle/astar.py`	**A*, Manhattan and Euclidean heuristics (new)**
`puzzle/visualize.py`	traceable text output (every step printed as a board)
`main.py`	command-line driver, timing, comparison table
`gui.py`	**graphical interface (new)**
`tests/test_algorithms.py`	24 automated tests

2 Algorithms

2.1 Breadth-first search

BFS expands nodes level by level using a FIFO `deque`; a `visited` set avoids repeats and a `parents` dictionary {`state: (parent, move)`} rebuilds the path. With unit costs, the first time the goal is reached is at the minimum depth, so BFS is *complete and optimal*. Cost: $O(b^d)$ time *and* memory, which is its weakness (179,170 expansions for a 28-move board).

2.2 Depth-first search

DFS uses a LIFO stack (a Python list) and a visited set. It dives down one branch before trying the next. It is complete here only because of the visited set, but it is *not optimal*: it returns the first solution it meets, which is usually enormously long (see the DFS rows in the results: 31 moves for a 3-move puzzle, 86,930 moves for seed 7).

2.3 Iterative-deepening DFS (IDS)

Idea. Run a depth-limited DFS with limit $0, 1, 2, \ldots$ until the goal is found. Memory is only $O(d)$ like DFS, yet the first goal found is the shallowest one like BFS, so IDS is complete and optimal for

unit costs.

Data structures. An explicit stack of `(state, depth, children-iterator)` (no recursion, so no recursion-limit problems); a dictionary `parents` holding *only the states on the current root-to-node path*, used both for cycle checking and for rebuilding the answer.

Why no global visited set. A state may be reached first by a long path inside one iteration and later by a shorter path. A global set would block the shorter one and IDS could miss solutions within the limit. Therefore cycles are checked against the *current path only*, and this set is rebuilt in every iteration.

Nodes expanded are summed across iterations. Each iteration re-expands the shallow levels, and all that real work is counted; the counter is never reset. For a solution at depth d the total is roughly $\sum_{i=0}^{d} b^i$, only a constant factor above BFS's last level (*the cost of repeated work*). Because the path-only cycle check cannot detect that the same state is reachable by many different paths, IDS expands far more nodes than BFS in practice on this puzzle (18 million for seed 7 against 179 thousand), trading time for $O(d)$ memory.

IDS pseudocode

```
ids(initial, max_limit=31):
    for limit in 0..max_limit:
        goal = depth_limited(initial, limit) # counters are NOT reset
        if goal: return path
    return failure

depth_limited(state, limit): # DFS, stack, path-only cycle check
    pop node; if goal -> return
    if depth == limit -> do not expand
    expanded += 1
    for child in neighbours: if child not on current path: push
```

31 is the largest optimal solution length of any solvable 8-puzzle board, so a solvable board is always found.

2.4 A* search

A* always expands the frontier node with the lowest $f(n) = g(n) + h(n)$, where g is the moves so far and h a heuristic estimate of the moves left.

Data structures.

- `frontier`: a binary min-heap (`heapq`) of `(f, counter, g, state)`. The unique `counter` is a tie-breaker so equal-f entries never compare states and ties pop in FIFO order.
- `best_g`: dictionary with the cheapest g found for each state (a worse duplicate is never pushed).
- `closed`: set of expanded states; stale heap entries are skipped when popped.
- `parents`: dictionary for path reconstruction.

The goal test is done when a node is popped, which (with an admissible heuristic) guarantees optimality. Because both heuristics are *consistent* (below), a closed set never discards a better path.

A* pseudocode

```
push (h(start), 0, start)
while frontier:
    f, g, s = pop min f # heap
    if s in closed: continue # stale entry
    closed.add(s)
    if s is goal: return path(s)
    expanded += 1
    for each child c of s:
        if c in closed: continue
        if g+1 < best_g[c]: # found a cheaper way to c
            best_g[c] = g+1; parent[c] = s
            push (g+1 + h(c), tiebreak, c)
```

3 Heuristics: Manhattan vs. Euclidean

Both sum over the **eight tiles only** (the blank is excluded — counting it would overestimate and break admissibility), comparing each tile's current cell (r, c) with its goal cell (g_r, g_c):

$$h_M = \sum_{t \neq 0} \big(|r_t - g_{r,t}| + |c_t - g_{c,t}| \big), \qquad h_E = \sum_{t \neq 0} \sqrt{(r_t - g_{r,t})^2 + (c_t - g_{c,t})^2}.$$

Admissibility. A tile moves one cell per move, horizontally or vertically, so tile t needs at least $|\Delta r| + |\Delta c|$ moves, and each move moves only one tile. Hence h_M never overestimates. Since $\sqrt{\Delta r^2 + \Delta c^2} \leq |\Delta r| + |\Delta c|$ for every tile, $h_E \leq h_M \leq h^*$, so h_E is admissible too. Both are also *consistent* (one move changes one tile's distance by at most 1), which is why the closed set is safe.

Which is "more admissible"? Both are admissible; neither ever overestimates, which I also verified in the tests against BFS's true costs. What differs is *informedness*: $h_E \leq h_M$ on every state, so Manhattan *dominates* Euclidean — it is closer to the true cost and the better heuristic. A* with a dominating heuristic expands no more nodes, and this is what the measurements show.

Table 1: A* heuristic comparison (median of 3 runs; both heuristics returned optimal paths of equal cost).

Initial state	Cost (M/E)	Exp. Manhattan	Exp. Euclidean	Euclid/Manh.	Time M	Time E
`1,2,5,3,4,0,6,7,8`	3 / 3	3	3	1.00×	0.0 ms	0.0 ms
`1,5,6,0,8,4,7,2,3`	23 / 23	762	1,402	1.84×	2.8 ms	5.8 ms
`2,3,1,0,8,7,6,5,4`	21 / 21	815	1,073	1.32×	2.9 ms	4.5 ms
`4,6,5,0,7,8,3,1,2`	25 / 25	3,802	5,547	1.46×	15.9 ms	26.5 ms
`1,6,7,4,0,8,3,2,5`	28 / 28	14,592	20,708	1.42×	66.5 ms	106.3 ms

Euclidean expands 1.3–1.8× more nodes (once the board is non-trivial) than Manhattan on the hard boards, while the path costs are identical because both are admissible and A* is optimal. Even Euclidean expands 8–75× fewer nodes than BFS (e.g. 20,708 vs. 179,170 on seed 7, in about a quarter of BFS's time).

4 Results for all algorithms

Five initial states: the lab's sample, and random solvable boards generated by `random_state(seed)`. Times are the median of 3 runs (seed 7 / IDS: a single 30.7 s run). "Depth" is the depth of the goal node found; for DFS the cost equals the (very long) path it returns.

Table 2: Cost, nodes expanded, search depth and running time.

State	Algorithm	Cost	Nodes expanded	Depth	Time
sample	BFS	3	10	3	0.0 ms
	DFS	31	31	31	0.1 ms
	IDS	3	8	3	0.0 ms
	A* Manhattan	3	3	3	0.0 ms
	A* Euclidean	3	3	3	0.0 ms
seed 3	BFS	23	105,554	23	217.0 ms
	DFS	31,729	32,591	31,729	61.6 ms
	IDS	23	1,092,696	23	1.85 s
	A* Manhattan	23	762	23	2.8 ms
	A* Euclidean	23	1,402	23	5.8 ms
seed 5	BFS	21	58,851	21	116.7 ms
	DFS	51,001	52,722	51,001	106.1 ms
	IDS	21	336,483	21	561.3 ms
	A* Manhattan	21	815	21	2.9 ms
	A* Euclidean	21	1,073	21	4.5 ms
seed 11	BFS	25	151,977	25	332.3 ms
	DFS	105,229	114,859	105,229	265.5 ms
	IDS	25	3,497,005	25	5.93 s
	A* Manhattan	25	3,802	25	15.9 ms
	A* Euclidean	25	5,547	25	26.5 ms
seed 7	BFS	28	179,170	28	384.4 ms
	DFS	86,930	92,132	86,930	206.2 ms
	IDS	28	18,147,655	28	30.72 s
	A* Manhattan	28	14,592	28	66.5 ms
	A* Euclidean	28	20,708	28	106.3 ms

States: sample = `1,2,5,3,4,0,6,7,8`; seed 3 = `1,5,6,0,8,4,7,2,3`; seed 5 = `2,3,1,0,8,7,6,5,4`; seed 11 = `4,6,5,0,7,8,3,1,2`; seed 7 = `1,6,7,4,0,8,3,2,5`.

4.1 Observations

- **Optimality:** BFS, IDS and both A* variants always returned the same (optimal) cost; DFS did not.
- **Nodes expanded:** A* Manhattan $\ll$ A* Euclidean $<$ BFS $\ll$ IDS. On seed 11, Manhattan needs 3,802 nodes against BFS's 151,977 and IDS's 3,497,005.
- **Memory:** IDS and DFS keep only a path (or stack); BFS and A* store every generated state.
- **IDS:** its repeated work is the reason it is slowest in time, but it is the only optimal algorithm with linear memory.

4.2 Paths to the goal (blank moves: U/D/L/R)

sample, A* Manhattan (3 moves): U L L

sample, A* Euclidean (3 moves): U L L

seed 3, A* Manhattan (23 moves): R D R U U L D L D R U R D L U L D R U R U L L

seed 3, A* Euclidean (23 moves): U R R D L D R U L U L D R D L U R U R D L U L

seed 5, A* Manhattan (21 moves): U R R D D L U L U R D R U L D R D L U U L

seed 5, A* Euclidean (21 moves): U R R D D L U L U R D R U L D R D L U U L

seed 11, A* Manhattan (25 moves): R D R U L D L U U R D R U L D L U R D D L U R U L

seed 11, A* Euclidean (25 moves): D R U U R D D L U R U L D L D R R U U L L D R U L

seed 7, A* Manhattan (28 moves): R D L U R U L D R D L U U R D L L D R U U L D R D L U U

seed 7, A* Euclidean (28 moves): R D L U R U L D R D L U U R D L L D R U U L D R D L U U

DFS paths contain thousands of moves; the full list is printed by `python main.py --state ... --algo dfs`.

5 Sample run (step by step)

Command: `python main.py --state 1,2,5,3,4,0,6,7,8 --algo astar-manhattan`. The tile that just moved is shown in parentheses.

```
Initial state: (1, 2, 5, 3, 4, 0, 6, 7, 8)

===== astar-manhattan =====
Solution found: 3 moves
Path: Up -> Left -> Left

Step 0 (initial)
+---+---+---+
| 1 | 2 | 5 |
+---+---+---+
| 3 | 4 | |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+

Step 1: blank moves Up
+---+---+---+
| 1 | 2 | |
+---+---+---+
| 3 | 4 |(5)|
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+

Step 2: blank moves Left
+---+---+---+
| 1 | |(2)|
+---+---+---+
| 3 | 4 | 5 |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+

Step 3: blank moves Left (goal)
+---+---+---+
| |(1)| 2 |
+---+---+---+
| 3 | 4 | 5 |
+---+---+---+
| 6 | 7 | 8 |
+---+---+---+

algorithm cost expanded depth max_depth time(s)
------------------------------------------------------------
astar-manhattan 3 3 3 3 0.0001
```

All five algorithms on this state: BFS expands 10 nodes, DFS 31 (cost 31), IDS 8, A* Manhattan 3, A* Euclidean 3 (cost 3). Initially $h_M = 3$ and $h_E = 3.0$, equal to the true cost, so A* walks straight to the goal.

6 Extra work

- **Modern GUI** (`python main.py --gui`): animated sliding tiles, tiles turn green when in place, play/pause/step/scrub, speed control, keyboard shortcuts, click-to-move, shuffle, algorithm selector, a "Compare all" table with log-scale bars and clickable rows to replay any algorithm's solution, a time limit so slow runs are stopped, and a solvability warning. The Python code does the search; the browser only draws.
- Solvability check, parsing and validation of user input.
- 24 automated tests (`python -m pytest -q`): optimality against BFS on random boards, valid-path checks, heuristic values, admissibility, $h_E \leq h_M$, edge cases (already solved, unsolvable).

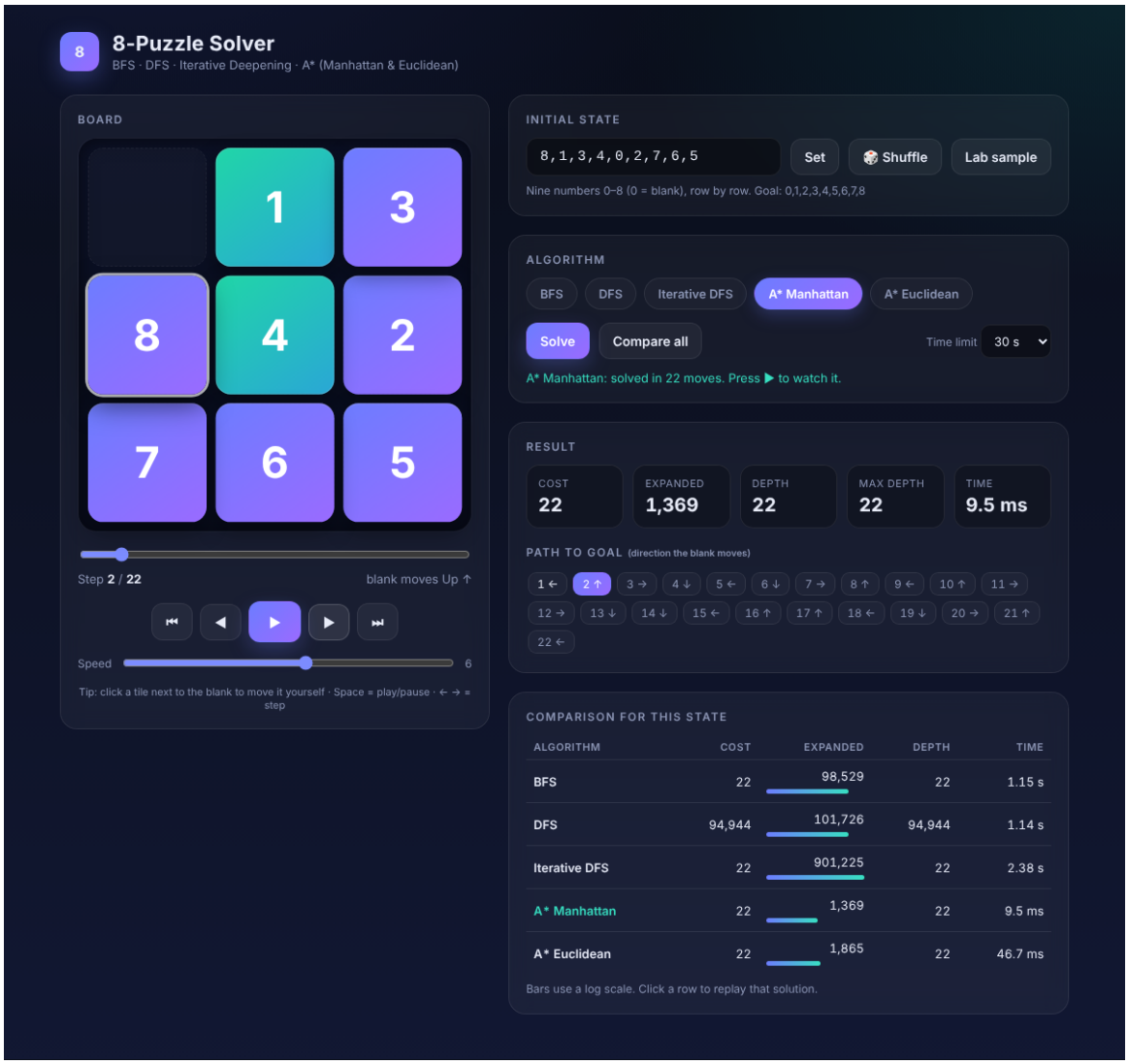


Figure 1: The GUI after "Compare all" on `8,1,3,4,0,2,7,6,5`, replaying the 22-move A* Manhattan solution.

7 How to run

```
python main.py --state 1,2,5,3,4,0,6,7,8 --algo all # all algorithms, boards printed
python main.py --random --seed 7 --algo all --no-trace # table only (IDS takes ~30 s)
python main.py --gui # graphical interface
```

```
python -m pytest -q # tests
```